

A Hybrid Biomedical Image-Analysis Pipeline for Fluorescence-Microscopy Nuclei

Github Link: <https://github.com/thihansas/biomedical-image-analysis>

1 Overview of methods

The dataset used in this study is synthetic fluorescence microscopy dataset with DAPI-like nuclear staining. It includes 112 RGB images (size 256×256 pixels). I did a 80/20/12 split for the training, validation and test sets, and also generated four different versions of the test data for the robustness experiment. Before carrying out the analysis, I converted the images to grayscale and resized them to 256×256 pixels. The resizing step is also included in the code so that the pipeline can still be used if the size of the input images changes in the future.

I formed the process as a four-stage pipeline. In the first task I began by looking at the images using a sample grid and an intensity histogram before having llama3.2-vision analyse them via Ollama. I compared a basic prompt with one that was more constrained and based on a schema and also carried out the same prompt three times to check whether the answers were consistent. In task two I implemented a more conventional image-processing method which involved Otsu thresholding, morphological cleaning, connected-component labelling and regionprops_table, only the resulting measurements were then passed on to a smaller local LLM, not the original images. Task three was concerned with segmentation and used a compact U-Net having four encoder/decoder stages, a base width of 16 and approximately 1.94 million parameters. Finally, in task four the trained U-Net, the region measurements and the JSON fields generated by code were all combined with a narrative that had been produced by an LLM for the 12 test images that had not been seen before. A single design decision applies throughout the entire pipeline, any numeric value that might affect the interpretation is calculated in code and not left to the LLM.

2 Task 1 - data preparation and VLM description

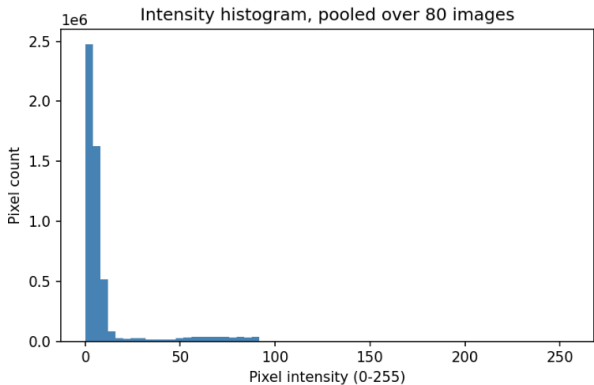


Fig. 1. Pooled pixel-intensity histogram for the 80 training images.

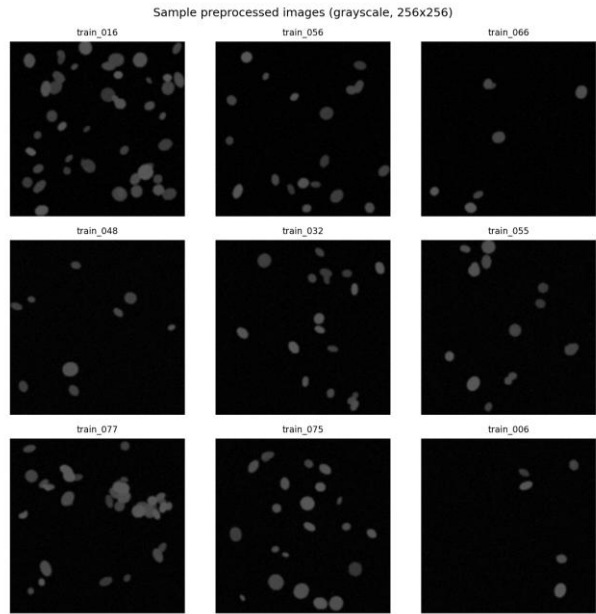


Fig. 2. Nine randomly selected training images after conversion to grayscale and resizing to 256×256.

The first prompt created an issue I did not expect. But the model didn't just describe what could be seen, it said the cells "look healthy and normal" and there were "no visible signs of damage". That is a judgment, not a description. It is inappropriate for a system that is simply supposed to analyze an image. So, I modified the prompt to ask the model to only describe the image and to explicitly return "uncertain" when it could not make a confident observation.

Optimised prompt used for the reported VLM results

"You are an assistant that provides objective, descriptive observations about a biomedical research image, for educational purposes only. You are NOT a diagnostic tool: you must not name a disease, give a diagnosis, or suggest treatment. Describe only what is visually observable."

Report: modality, tissue_type, notable_features (1-2 sentences, purely descriptive), image_quality (focus/contrast/noise). If not confident about any field, write "uncertain" instead of guessing.

Respond with ONLY a single JSON object: {"modality": "...", "tissue_type": "...", "notable_features": "...", "image_quality": "..."}"

Example output: {"modality": "fluorescence microscopy", "tissue_type": "uncertain", "notable_features": "Bright blue spots are scattered across the image, possibly representing cellular structures or other biological features.", "image_quality": "uncertain"}

Even without changing anything about the input image or prompt, I found the model to be inconsistent. When I ran the same prompt 3 times on the same image, the tissue_type was returned as "uncertain", then "cell culture", then "uncertain" again. There was a change to the notable_features field and image_quality result's wording. Because the results changed between runs, I decided not to rely on a single VLM response and kept the LLM's role smaller in the later tasks.

3 Task 2 - classical features and numbers-first interpretation

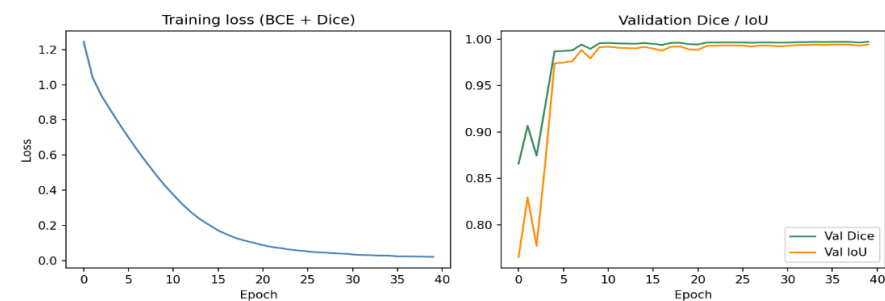


Fig. 3. Training loss (BCE+Dice) and validation Dice/IoU over 40 epochs

As an example, the Otsu-based processing of train_004 detected 37 objects. The region table contains measurements such as area, eccentricity, solidity and mean intensity for each component.

label	area	ecc.	solidity	mean int.
1	267	0.761	0.950	66.8
3	478	0.821	0.746	64.8
4	2036	0.826	0.565	73.5
7	102	0.650	0.953	52.0

Numbers-first interpretation prompt

“You are an assistant that interprets a table of image-analysis measurements... You are NOT given the image itself, only these numeric measurements, and must not invent visual details... You are NOT a diagnostic tool.”

Measurements summary: {n_objects, area/eccentricity/solidity/mean_intensity/extent stats}

1. Write ONE short paragraph (3-4 sentences) on count, density, size spread, shape regularity.

2. Output JSON: {"n_objects": int, "density_class": "sparse|normal|dense", "shape_regularity": "regular|irregular", "quality_flag": "ok|uncertain"}. If ambiguous, use "uncertain" rather than guessing.

LLM json: {"n_objects": 37, "density_class": "sparse", "shape_regularity": "irregular", "quality_flag": "uncertain"}

Code json: {"n_objects": 37, "density_class": "normal", "shape_regularity": "irregular", "quality_flag": "ok", "mean_area": 360.9}

The two outputs were different even though they were based on the same measurements. The rule used in the code is that less than 15 objects is sparse, 15-40 objects is normal, and more than 40 objects is dense. The image has 37 objects and the code correctly identifies it as normal density. The LLM came back sparse instead, despite being given the correct count. That is, the model made a simple numerical reasoning error when it wasn't even looking at the image. So, I take the value generated by the code as the ground truth and use the LLM only for interpretation and phrasing.

Running the same image through Otsu and regionprops again should give the same measurements, so I can check the results and trace them back to the image-processing steps. The LLM is only used to turn these measurements into a more readable description. It is not used to measure anything from the image. The VLM description was useful for getting an initial idea of the image, but the repeated runs showed that the result could change.

4 Task 3 - U-Net segmentation

For Task 3, I trained the U-Net for 40 epochs using the 80 training images and evaluated it on the 20-image validation set. The training and validation curves in Fig. 3 suggest that most of the improvement happened fairly early, with the curves beginning to level off around epochs 15–20.

Split	Method	Mean Dice	Mean IoU
val (n=20)	Otsu (classical)	0.976	0.953
val (n=20)	U-Net (BCE+Dice, 40 ep.)	0.997	0.994

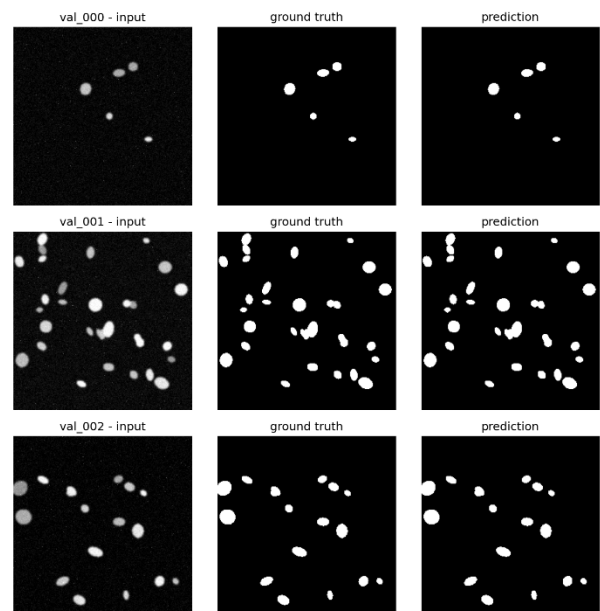


Fig. 4. Input / ground-truth / U-Net prediction for 3 validation images (val_000–val_002).

Dice and IoU are both overlap measures between the predicted mask (P) and the ground-truth mask (G). Dice is calculated as $2|P \cap G| / (|P| + |G|)$, while IoU is $|P \cap G| / |P \cup G|$. IoU is a little stricter when the predicted region varies in size or around the boundary. The mean validation scores were 0.997 for Dice and 0.994 for IoU, so the predicted masks were very close to the ground-truth masks on this dataset. The individual Dice scores were also tightly grouped between 0.995 and 0.999. The lower results occurred on images with more small nuclei or touching nuclei, such as val_017, val_005 and val_012. One limitation is that the U-Net produces a single binary mask rather than separate labels for each nucleus. As a result, two touching nuclei can be merged even when the overall pixel-level Dice score is very high.

5 U-Net vs Otsu, and the hybrid pipeline

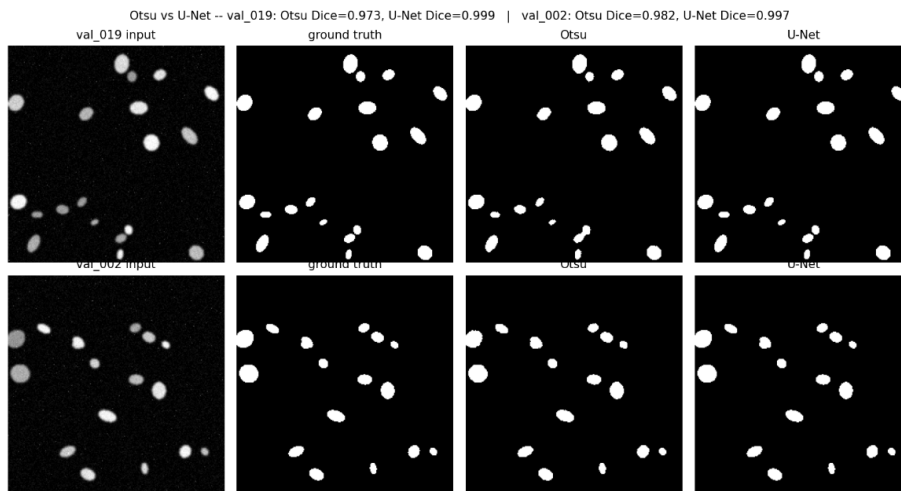


Fig. 5. U-Net's largest margin over Otsu (val_019, touching/medium-contrast nuclei) and its smallest (val_002, large well-separated bright nuclei).

The U-Net performed at least as well as Otsu on every validation image. Its mean Dice was 0.997 compared with 0.976 for Otsu, while the mean IoU was 0.994 compared with 0.953. The biggest difference was on val_019, where the U-Net achieved a Dice score of 0.999 compared with 0.973 for Otsu. This image contains touching nuclei with medium contrast, which makes a single global threshold less effective and leads to rough edges and small fragments. The difference was much smaller on val_002, where the nuclei were large, bright and well separated. In that case, the simpler Otsu method was already able to produce a reasonably good mask.

Task 4 example record (test_000, unseen)

```
{"image_id": "test_000", "n_objects": 8, "mean_area": 191.0,
"density_class": "sparse", "quality_flag": "ok", "narrative":
"The test_000 image contains 8 objects with varying sizes, as
indicated by a standard deviation of 111 pixels in their areas.
These objects appear to have a range of shapes ... suggesting
some elongation or irregularity. However, the mean solidity and
```

extent values suggest that most ... are roughly convex and well-defined ..."}

I then ran the same hybrid process on all 12 unseen test images. The sequence was U-Net segmentation, regionprops measurements, code-generated JSON fields and finally an LLM-generated narrative. The results were saved to outputs/task4_hybrid_records.csv, with one row for each test image. The detected object count ranged from 8 to 43, covering the sparse, normal and dense categories.

"You are an assistant that writes a short, objective narrative for a biomedical research image, for educational purposes only. You are NOT a diagnostic tool. You are given only numeric measurements below, not the image itself — do not invent visual details beyond what the numbers imply. Image id: {image_id}. Measurements from automatic segmentation: {summary_text}. Write ONE short paragraph (3-4 sentences)... If the numbers are ambiguous, say so rather than guessing. Plain text only, no JSON."

The LLM can still produce output that's misleading in several ways. For example when I ran the VLM times in Task 1 it gave me different descriptions for the same image.. In Task 2 the model even got the density category wrong when it was given the correct object count. I did not just assume that a better prompt would fix these problems. Instead I tried to reduce these issues.

I made the prompts allow the LLM to say that it is "uncertain". I also made it explicitly state that it is not a tool for diagnosis.. I kept the interpretation model away from the original pixels in Tasks 2 and 4. I added a step for malformed JSON too.. The most important thing is that the measurements are calculated by code. This means that the numbers can be reproduced and checked to make sure they are sensible. We can also trace them back to the image-processing operation that produced them.

Robustness extension

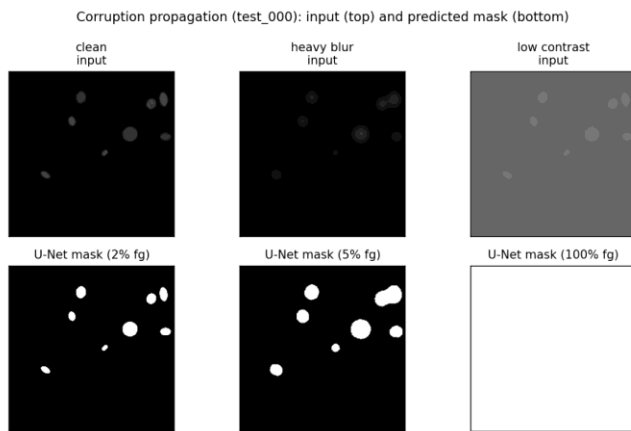


Fig. 6. Corruption propagation on test_000. Blur weakens object boundaries but still gives a usable mask.

For the robustness extension I took two test images and corrupted them in two ways: I added a lot of blur. I rescaled them to have low contrast. Then I ran these images through the same pipeline. When I added blur, the pixel standard deviation fell by 40 to 44 percent. The mask Dice against the ground truth dropped from around 0.997 to 0.65. The object counts also fell. Not as much. The low-contrast case was much worse. The U-Net classified the image as foreground which gave Dice values of around 0.05 to 0.36. The Regionprops treated the image as one object with an area of 65,535 pixels, which is the same as the total number of pixels in the image. What is interesting is that the LLM narrative still sounded okay. It described the result as a elongated oval-shaped object and did not say that the segmentation had failed. The structured measurement made it much easier to spot the problem. In this case, the numbers made the segmentation problem much easier to spot than the generated text did.

The dataset is synthetic and quite clean, so I would not expect these high segmentation scores to transfer directly to real microscopy images. The U-Net also does not separate nuclei at the instance level. The VLM experiments showed that the language model can be inconsistent. The robustness experiment showed that it can produce a narrative even when the underlying segmentation is clearly wrong. For these reasons I do not think that the current system is suitable for use in a clinic. The next thing I would do is retrain and test the model using a dataset with expert annotations. It would be ideal to use images from a clinical site that have been held out for testing. I would also add checks to the code-generated JSON, such as flagging an object area that is greater than or equal, to the whole image area before any narrative is shown to a user. This would help to make the system more reliable. Overall, the experiments showed

that LLM should not be relied on for measurements or decisions about the image. It was more useful for turning the measurements produced by the code into a short description. The VLM was useful for an initial description, but the repeated runs showed that its output was not always consistent.

References

- Ronneberger, O., Fischer, P., Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. MICCAI.
- Otsu, N. (1979). A Threshold Selection Method from Gray-Level Histograms. IEEE Trans. Systems, Man, and Cybernetics, 9(1), 62–66.
- van der Walt, S. et al. (2014). scikit-image: image processing in Python. PeerJ, 2:e453.
- Sørensen, T. (1948). A method of establishing groups of equal amplitude... Biol. Skr., 5, 1–34. (Dice/Sørensen coefficient.)
- Meta AI (2024). Llama 3.2 Model Card (incl. vision variants). Served locally via Ollama (ollama.com).
- Caicedo, J. C. et al. (2019). Nucleus segmentation across imaging experiments: the 2018 Data Science Bowl. Nature Methods, 16, 1247–1253.